



Confide

Agency collaboration with client protection built in

The chat is the surface. The control layer is the product.

Platforms · Web · Desktop (Windows, macOS, Linux) · Mobile (Android, iPhone)

Status · Phase 1 complete — 68 of 70 planned tasks delivered

Document · Full project documentation, all platforms

Date · 28 August 2026

Contents

- 1** What Confide is, and the problem it solves
The business case, in plain terms
- 2** The three guarantees
Rules the system cannot break, even with a software bug
- 3** How message screening works
The detection engine — the heart of the product
- 4** Roles and who can do what
Admin, team member, client
- 5** The moderation workflow
Held, approved, blocked, escalated
- 6** The three platforms
Web, desktop and mobile — what each is and why it exists
- 7** Technology choices and the reasoning
Every major component, and why it was chosen
- 8** Security and data protection
How the guarantees are enforced technically
- 9** The audit trail
Evidence that stands up after the fact
- 10** Notifications
Three delivery routes, and when each applies
- 11** Quality and testing
What is verified automatically, and how often
- 12** Deployment and operations
Where it runs, how releases work
- 13** Known limits and what comes next
Stated openly, so nothing is discovered mid-pilot

HOW TO READ THIS DOCUMENT

Sections 1–6 are for anyone. Sections 7–12 go into technical detail for engineering review. Section 13 is deliberately candid about what the product does *not* do — that section matters most in a client conversation, because it is the difference between a promise and a commitment.

1. What Confide is, and the problem it solves

A messaging platform for agencies working with clients, where the messaging is the visible part and the protection is the product.

The problem

An agency introduces a client to a contractor, or a specialist, or a team member. They collaborate in a shared chat. Then someone shares a phone number or an email address, the conversation moves to WhatsApp, and the next project is arranged without the agency in the room.

This is the single largest revenue leak in agency work, and it is invisible: by the time it is noticed, the relationship has already moved. Ordinary chat tools cannot help, because from their point of view nothing unusual happened — someone simply sent a message.

What Confide does

Confide is a group chat platform where **every message is inspected by the server before anyone can read it**. If a message contains contact information — a phone number, an email address, a WhatsApp link, a payment handle — it does not get delivered. It waits for an administrator to approve or block it, and the decision is recorded permanently.

THE CORE GUARANTEE

A message containing contact information cannot reach its recipient unless a human administrator releases it. This is not enforced by the app politely declining to send — it is enforced by the database, which does not grant the browser permission to create a message at all.

What that gives the agency

- **Prevention** — contact details are stopped before they arrive, not flagged after the fact.
- **Visibility** — the administrator sees every attempt, including the ones they approve.
- **Evidence** — a permanent, tamper-evident record of who tried to share what, and when. This is what makes a non-circumvention agreement enforceable rather than decorative.
- **Identity control** — clients see the names the agency assigns, not the personal identities of the people doing the work.

What the client experiences

An ordinary, modern chat application. Groups, replies, file sharing, formatting, notifications, read receipts. The screening is invisible unless it triggers — and when it does, the sender sees a clear "pending review" marker on their own message rather than a silent failure.

2. The three guarantees

These are not features that could be switched off or bypassed by a bug in the application. They are enforced by the database itself.

Guarantee 1 — No private messages. Groups only.

There is no direct-message feature and no private channel. Every conversation is a group, and only an administrator can create one. A group of two people is possible — but only because an administrator made it, and it is screened, audited and readable by that administrator exactly like every other group.

Why it matters: an unobserved channel between two members is precisely where an exchange of contact details would happen. There is no such channel to find, and none that a member can open.

Guarantee 2 — Identity is controlled by the agency.

Members cannot change their own display name. When an administrator invites someone, the administrator sets the name the others will see. A member who wants a different name submits a request, which an administrator reviews.

Why it matters: without this, a member could simply rename themselves to their phone number or email address and bypass the entire screening system in a single move.

Guarantee 3 — Archive is not deletion, and the record is permanent.

Groups are archived, never erased. The audit log physically rejects any attempt to modify or delete an entry — including from the application's own administrative account. Each entry is cryptographically linked to the one before it, so removing or altering a single line breaks a chain that the system verifies nightly.

Why it matters: evidence that can be quietly edited is not evidence. This is what allows the record to be handed to a lawyer.

WHY ENFORCE AT THE DATABASE LEVEL

Application code changes constantly, and every change is a chance to introduce a mistake. These three rules are enforced one layer below the application, so a bug in a screen, a route or a future feature cannot violate them. Each is covered by an automated test that runs on every change.

3. How message screening works

The detection engine is the component the entire product rests on. It reads text and decides one of three things.

Outcome	What happens
Allow	Delivered immediately. Nothing recorded beyond the ordinary message record.
Flag	Delivered immediately, but the finding is recorded for the administrator. Used where something was detected but is very unlikely to be a genuine attempt — an automated email from a service, for example.
Hold	Not delivered. Waits in the moderation queue. The recipient sees nothing at all until an administrator approves it.

WHY THREE OUTCOMES AND NOT TWO

A system that only blocks or allows must choose between missing real attempts and freezing ordinary work. The middle outcome is what makes the product usable: a pasted error log containing a service email address is recorded but delivered, while a personal address is held.

What it detects

Twenty-eight separate detection rules across five categories:

Category	Examples caught
Phone numbers	International (+92 300 1234567), national (03001234567), spaced digits, digits spelled out in words from seven upwards, and numbers glued to text (call447304250183)
Email addresses	Standard, and disguised forms — john [at] gmail [dot] com , john@gmail.com , j o h n @ ...
Messaging links	WhatsApp, Telegram, Signal, Instagram, Snapchat, Discord, Skype, LinkedIn, Facebook
Payment handles	PayPal.me, Wise, Payoneer, IBAN bank details
Crypto wallets	Bitcoin (both formats), Ethereum, Tron

A short run of digits is not a phone number: "one two three", "meet at 3pm" and "v2.0.1" deliver untouched. Section 3b covers rooms that want even those held.

Where the check happens

Screening runs on the server, before a message row exists. It is not a check the app performs on its way to the database — it is a gate the message must pass to *become* a message at all. A modified client, a direct API call or a replayed request all arrive at the same place, so there is no route into a conversation that skips it.

3a. How the engine resists evasion

Anyone determined to pass a number through will try to disguise it. The engine runs in layers, each closing a different class of disguise.

The five stages, in order

Stage	What it does
1. Normalise	Text is reduced to a canonical form: look-alike letters from other alphabets become their Latin equivalents, and invisible characters are stripped.
2. Match	Each rule reads the form it needs: most the normalised text, wallet addresses the original, where capitals carry meaning.
3. Merge	Overlapping findings of the same kind collapse to the strongest one.
4. Allowlist	Known-harmless senders drop from hold to flag: delivered, still recorded.
5. Decide	Each finding carries a confidence score. At or above the hold threshold the message stops; below it, it delivers flagged.

Defeating disguises

A Cyrillic "o" in an email address, or a zero-width space inside a phone number, is removed before matching. Formatting is stripped and the message re-checked, so a number split with bold markers still reads as one number. Letters standing in for digits — `o` for zero, `l` for one — are folded back inside runs that are mostly digits.

Phone numbers are validated, not just matched

A run of digits is checked against the international numbering standard to see whether it is a real number in *any* country, using per-country length and prefix rules. A bare number from Pakistan, the UAE, the UK or India is caught with no configuration; an invoice or order reference of the same length is not.

The allowlist is not a blind spot

Seventeen tooling domains — GitHub, Stripe, Figma, Sentry — and machine senders such as `noreply@` are demoted rather than held, because they appear constantly in real work. Free consumer providers are deliberately absent: that is where contact exchange happens. A demoted finding is still recorded.

Measured accuracy

The engine is tested against a corpus of real and adversarial examples on every change, and every evasion found joins that corpus permanently, so the same trick cannot work twice. Two thresholds are enforced automatically, and a change breaching either cannot be merged:

279	100%	0%	28
test cases in the corpus	of contact info caught	false alarms	detection rules

3b. Screening strictness, per group

Different rooms need different rules. A client-facing room and an internal team room should not be screened identically.

An administrator sets the level from the group's menu. It applies from the next message, with no technical work required.

Level	Holds	Ordinary messages held
Contact info only default	Phone numbers, emails, messaging links, payment handles, crypto addresses	None measured
Long numbers too	The above, plus any number of seven digits or more	About 27%
Any number at all	The above, plus every digit — including "3pm" and "v2.0.1"	About 66%

RECOMMENDATION

Use the strictest level on client-facing rooms only. The percentages above are measured, not estimated — at the strictest setting roughly two-thirds of ordinary conversation is held for review, which is appropriate for a room where no number should ever be exchanged and unworkable for an internal team room.

What cannot be switched off

The settings control whether a finding *holds* a message. They cannot disable detection itself, and they cannot disable the audit trail. Contact information is always detected and always recorded; a group may only choose whether it waits for review or is delivered with a note.

4. Roles and who can do what

Two things decide what someone may do: their role in the workspace, and whether they manage a particular group.

Role	Can do	Cannot do
Administrator the agency	Create groups, invite people, set display names, approve or block held messages, set screening strictness, archive and delete groups, read the full audit trail and export it	Modify or delete audit entries. Delete a group without archiving it first.
Team member staff, contractors	Read and write in the groups they belong to, share files, reply, mention colleagues	Change their own display name, create groups, invite anyone, see groups they are not in. Sees the moderation queue only if made that group's manager
Client	Everything a team member can, in their own groups: read and write, share files, reply, mention the other people in the group, and see who else is in it	Everything a member cannot, plus: be a group's manager, see other members' internal nicknames, or see the moderation queue under any circumstances

What each person sees of the others

A visibility layer decides field by field what one role may see about another. A client sees display name, role label and avatar. Team members see those plus internal nicknames. Real email addresses and phone numbers are stored in a table that the application's ordinary read permissions cannot access at all — not filtered in code, but unreadable at the database level.

ONE GROUP MANAGER

A group may have a designated manager who can approve or block held messages for that group only. A client can never hold this role, because the moderation queue displays exactly the contact details the holds exist to contain.

That restriction is a database trigger rather than a check in the application: the row promoting a client to manager cannot be written at all, so no route, script or future feature can create one by mistake.

Removal takes effect in seconds

When someone is removed from a group or workspace, their existing session is revoked rather than left to expire. Access tokens are short-lived by design, so a removed person cannot continue reading past that window even if they keep the application open.

5. The moderation workflow

- 1 **Someone sends a message.** It goes to the server. The browser has no permission to write a message directly — this is enforced by database privileges, not by application code.
- 2 **The server screens it.** The detection engine runs before any message record exists. Formatting is stripped and it is screened a second time, so disguises using bold or italics do not work.
- 3 **Clean messages are delivered instantly.** They appear for everyone in the group in real time.
- 4 **Flagged messages are held.** The sender sees "pending review" on their own message. Other members see nothing — not a placeholder, not a gap. The message does not exist for them.
- 5 **Administrators are alerted.** Their dashboard updates live, and an email is sent as a fallback if no administrator is currently online.
- 6 **An administrator decides.** They see the message with the detected text highlighted exactly as it was typed, and choose approve or block. Both outcomes are recorded permanently with who decided and how long it took.
- 7 **Approved messages are delivered** in their original position in the conversation, marked "released after review" so they are not missed by someone who has already scrolled past.

If nobody responds

After	What happens
30 minutes	The held message escalates to the group manager. Recorded in the audit trail as a system action.
8 hours	The message is released automatically, still flagged and still fully recorded, and marked as auto-released rather than approved by a person.

Both intervals are configurable per group. The reasoning for automatic release is that a chat tool which silently swallows messages overnight will not be used — and a tool that is not used protects nothing. The release is visible and attributed, never silent.

6. The three platforms

One product, one codebase, three ways to reach it. Everything below shares the same screening, the same data and the same audit trail.

THE ARCHITECTURAL DECISION BEHIND ALL THREE

The application itself is a web application. The desktop and mobile apps are native windows wrapping that same web application — the identical approach Slack and Discord use. The consequence is significant: **every improvement ships to all three platforms at once**, with no app-store review and no reinstall. Only changes to native behaviour require a new build — two so far, both for the desktop app.

Platform comparison

	Web	Desktop	Mobile
Install needed	No	Yes	Optional
Own icon & window	No	Yes	Yes
Notifications while closed	Yes	Yes	Browser-installed: yes Android app: not yet
Updates automatically	Yes	Yes, with a prompt	Content yes, app shell manually
Distribution	A web address	Installer download	Browser install, or app file

6a. The web application

The product itself. Everything else is a window onto this.

What it is

A modern web application that runs in any current browser on any device. There is nothing to install, updates are instant, and there is no version fragmentation — everyone is always on the current release.

What it contains

Screen	Purpose
Dashboard	The administrator's control surface: what is awaiting review, what was caught this week, whether the evidence chain still verifies, recent activity. Members and clients see a version suited to them.
Chat	Conversations, replies, file sharing, mentions, rich text formatting, read receipts, typing indicators, live presence.
Moderation	The queue of held messages, each showing the detected text highlighted, with approve and block.
Names	Requests from members to change their display name, for administrator review.
Audit	The full permanent record, searchable and filterable, with chain verification and export.

Design and usability

- Works on any screen from 360 pixels upward — every flow completes on a phone without sideways scrolling.
- Six colour themes, chosen per person and remembered across devices.
- Keyboard shortcut (⌘K / Ctrl+K) to jump to any page or group.
- Loading placeholders shaped like the real content, so nothing jumps as it arrives.
- Automatically refreshes itself when a new version is deployed, at a moment the user is not mid-sentence.

Installable from the browser

On both Android and iPhone the web application can be added to the home screen from the browser. It then has its own icon, opens full-screen with no browser bar, and receives notifications even when closed. This requires no app store, no download and no approval process — and on Android it is currently the only route that delivers notifications while the app is not running.

6b. The desktop application

Windows, macOS and Linux, built from one codebase.

What it is

A native desktop application — its own icon, its own window, its own place in the taskbar or dock. Inside that window runs the same web application, which is why it never falls behind: the moment a change is deployed, the desktop app has it.

Built with Tauri

Aspect	Detail
Framework	Tauri 2 — a Rust-based shell that uses the operating system's own browser engine
Why Tauri over Electron	Electron bundles an entire copy of Chrome into every application, producing installers of 100 MB and upwards. Tauri uses the browser engine already present on the machine, so the installer is a few megabytes and the application uses far less memory.
Current version	0.1.2
Platforms	Windows (.msi and .exe), macOS (.dmg, Intel and Apple Silicon), Linux (.deb and AppImage)

What the desktop app adds over a browser tab

- **Native notifications** — through the operating system, not the browser, which is more reliable and works when minimised.
- **Unread badge** — on the dock icon (macOS) or taskbar (Windows), showing what needs attention.
- **Single instance** — clicking the icon twice focuses the existing window instead of opening a second copy.
- **Remembered window position and size** between sessions.
- **External links open in the real browser**, not trapped inside the app window.
- **Real file downloads** into the Downloads folder, with a notification when finished.
- **Automatic updates** — checks after launch and every five minutes while open. A new version offers to install itself, verifies its signature, installs and restarts.

Security of the desktop bridge

The web application is given exactly two native capabilities: showing notifications and setting the window badge. Nothing else is reachable — no file system, no command execution — enforced by a permission file scoped to the application's own web address.

Release process

Publishing a version is a single tagged commit. An automated pipeline builds all three operating systems, signs the update artefacts and publishes the installers; installed apps find the release in minutes.

6c. Mobile

Two routes, and the simpler one is currently the better one.

Route 1 — Install from the browser (available today)

Open Confide in Chrome on Android or Safari on iPhone and choose "Add to Home Screen". The result has its own icon, opens full-screen without a browser bar, and — importantly — **receives notifications even when fully closed**.

This works on both Android and iPhone, requires no app store, no review process and no cost, and updates itself with every deployment.

Route 2 — The Android app file (built, in testing)

A genuine Android application, built from the same Tauri shell as the desktop app and distributed directly through the project's own releases rather than the Play Store.

Aspect	Detail
Distribution	Direct download and install — no Play Store listing, so no review process and no risk of rejection
Architecture	arm64 — every Android phone made since roughly 2017
Content updates	Automatic, with every deployment, exactly like the desktop app
App updates	Manual. Android does not permit an application to install itself silently, so a new version is downloaded and tapped. The app shows a notice when one is available.

THE ONE REAL GAP

Notifications while the Android *app* is fully closed do not work yet. Android's system for this requires an integration with Google's messaging service — free of charge, roughly two to three days of work. Until then, the browser-installed version handles this correctly, which is why it is the recommended route for the pilot.

iPhone

The browser-installed route works fully on iPhone today, including notifications. A native iPhone application is possible from the same codebase, but requires a Mac to build and an Apple Developer subscription at \$99 per year — a decision worth taking only if App Store presence becomes a requirement.

Mobile-specific engineering

Substantial work has gone into making the phone experience correct rather than merely functional: the message box collapses to a single row on a phone, the back gesture returns to the conversation list instead of closing the app, the keyboard no longer covers the message box, menus stay on screen, and the formatting toolbar scrolls sideways rather than wrapping.

7. Technology choices and the reasoning

Every significant component, what it does, and why it was chosen over the alternatives.

Component	Choice	Purpose and reasoning
Application framework	Next.js 16.3 React 19.2	Renders pages on the server for speed and search visibility, and provides the server-side routes where screening happens. One language and one codebase across the whole product.
Language	TypeScript 5	Catches whole classes of mistake before the code ever runs. On a product where a bug can mean a leaked phone number, this is a safety measure rather than a convenience.
Database	Supabase (PostgreSQL)	Provides the database, sign-in, live updates and file storage in one service. PostgreSQL specifically because its row-level security is what allows the three guarantees to be enforced below the application.
Styling	Tailwind CSS 4	Consistent design system, small output, and no separate stylesheet that can drift out of step with the interface.
Validation	Zod 4	Every piece of data arriving from a browser is checked against a strict schema before it is used. Nothing untrusted reaches the database unvalidated.
Phone validation	libphonenumber	Google's phone number library, checking numbers against real per-country rules. This is what lets a bare number from any of 245 countries and territories be recognised while invoice and order numbers are not.
Desktop & mobile shell	Tauri 2 (Rust)	Native applications for Windows, macOS, Linux and Android from one small codebase, using the operating system's own browser engine instead of bundling one.
Email	Resend	Invitations and administrator alerts.
Notifications	Web Push (VAPID)	Delivers alerts to closed and minimised browsers, encrypted end to end.
Hosting	Vercel	Deploys on every change, scales automatically, hosted in the Tokyo region.
Testing	Vitest, Playwright	Unit tests, database rule tests, and full user-journey tests in a real browser.

7a. How the pieces fit together

Three architectural decisions do most of the work. Each moves a guarantee out of the application code, where a bug could break it, and into a layer that cannot be talked out of it.

The path a message takes

Step	
1	Someone types a message — in a browser, or in the desktop or Android app, which run the same application.
2	It is sent to a server route. The browser is not permitted to write it directly.
3	The route establishes who is asking and whether they may write to that group.
4	The detection engine reads the text and returns allow, flag or hold.
5	Only then is a row written, with that outcome already attached to it.
6	Recipients receive it over a live connection that carries only what the database's own rules permit each person to see.

Two database identities, deliberately unequal

The browser connects with an identity that may *read* messages and nothing more. The server connects with a separate, privileged identity kept in a file that fails the build if it is ever imported into browser code. A leak of that key would expose every workspace, so the compiler prevents it rather than developer discipline.

One authorization choke point

Because the privileged identity bypasses the database's own row rules, every route that uses it first calls a single authorization function covering thirteen named actions — send a message, moderate one, add a member, archive a group, and so on. There is one place to read, one place to test, and no second route with its own interpretation of who may do what.

A request aimed at another agency's workspace is answered "not found" rather than "not allowed", because confirming that a group exists is itself a disclosure.

WHY THE BROWSER CANNOT WRITE MESSAGES

Every message must pass through a server route, and that route runs the detection engine before writing anything. The guarantee therefore does not depend on the application behaving correctly: someone with a valid login, developer tools open and full knowledge of the system still cannot insert a message that skipped screening — the database will refuse it.

8. Security and data protection

Separation between workspaces

Every table carries a workspace identifier and every access rule filters through it. An automated test generates the list of tables from the database itself and verifies, for each one, that a user in one workspace can read nothing belonging to another. A new table ships with this coverage or the build fails — there is no hand-maintained list to forget to update.

Contact details are unreadable, not merely hidden

Real email addresses and phone numbers live in a table the application's ordinary read permissions cannot access at all. This is not a filter applied in code that could be forgotten on one screen; the permission does not exist.

Files

- Stored in private storage. No file has a public address.
- Downloads are authorised individually and given a link valid for five minutes.
- Filenames are screened exactly like message text — a file named with a phone number holds the message carrying it.
- A virus-scanning step sits in the upload path, ready for a scanner to be connected.

Sign-in and sessions

- Email and password, with rate limiting per account and per network address.
- Sign-in responses are deliberately identical whether or not the account exists, so the login form cannot be used to discover who has an account.
- Access tokens last ten minutes and renew invisibly, so removing someone takes effect within that window rather than whenever they happen to sign out.
- Invitation links are single-use, expire after seven days, and the link itself is never stored — only an irreversible fingerprint of it.

Logging discipline

No message content, email address or phone number is written to server logs. A product whose value is confidentiality cannot leak conversation into a third-party logging service, so log entries carry identifiers only.

Recorded identity events

Sign-in, sign-out and password changes are all recorded in the permanent audit trail — so the question "who could have been in this account, and from when" has an answer.

9. The audit trail

The feature that turns a policy into something enforceable.

What is recorded

Every action of consequence: messages sent, held, approved, blocked, escalated and auto-released; groups created, archived and deleted; people invited, added and removed; names changed and name requests reviewed; files uploaded; sign-ins, sign-outs and password changes; screening settings changed, with both the previous and new values.

Why it cannot be altered

- The database rejects any attempt to modify or delete an entry — including from the application's own administrative account.
- Each entry carries a cryptographic fingerprint of the entry before it, forming a chain. Changing or removing any line breaks every fingerprint after it.
- The chain is verified automatically every night, and the result is shown on the dashboard. An administrator can also verify it on demand.

Why entries survive deletion

Audit entries store the names of people and groups as plain text copies rather than references. When a group is permanently deleted or a person leaves, the record still reads correctly years later. An entry that renders as "<deleted> did something in <deleted>" is not evidence.

Export for a dispute

The full timeline exports to a spreadsheet file, with three levels of detail chosen at export time:

Level	Contains
Timeline only	Who did what, and when. No message content.
Plus findings	Adds the exact text that was detected — the phone number or address itself. This is usually the fact a dispute turns on.
Plus full messages	Adds the complete message text.

Exports that include message content are themselves recorded in the audit trail, naming who exported, how much, and what was included. An evidence surface that cannot say who copied it is a weaker one.

10. Notifications

Three delivery routes, because no single one covers every situation.

Route	When it applies	Where it works
In-page	The application is open and visible	Everywhere
Operating system	The desktop app is running, possibly minimised	Desktop app
Web push	The browser is closed entirely	Web, and browser-installed mobile

How it behaves

- One control turns notifications on or off, and it governs all routes.
- Only one route fires per message — no duplicate alerts.
- Notification content respects the visibility rules: recipients see the display name they are permitted to see, not the sender's real identity.
- **Held messages notify nobody.** A message waiting for review has not been delivered, and a notification would defeat the hold entirely.
- Sound and an unread badge accompany the alert where the platform supports them.

Administrator alerts

When a message is held, administrator dashboards update live. If no administrator is currently online, an email is sent instead. Both are best-effort — the hold itself is already permanent in the database, so a missed alert delays review but never loses a message.

11. Quality and testing

Every change runs the full set automatically. A failure blocks the change from being merged.



What each layer checks

Layer	What it proves
Unit tests	Individual pieces behave correctly — the detection engine, the permission function, text normalisation, invitation tokens.
Database rule tests	The three guarantees hold at the database level: workspace separation, no write permissions for browsers, the audit log rejecting changes, deletion rules.
Screening corpus	Accuracy against real and adversarial examples, with a minimum catch rate and a maximum false-alarm rate enforced automatically.
Journey tests	The entire product working end to end in a real browser: invite someone, accept, send a phone number, confirm the sender sees "pending review", confirm the recipient receives nothing, approve it, confirm delivery, confirm the audit trail records the whole chain.

THE TEST THAT MATTERS MOST

One automated test performs the product's entire reason for existing: that a message containing a phone number reaches no recipient until an administrator approves it, and that every step is recorded. If that test fails, nothing ships.

The screening corpus, specifically

Two limits are enforced on every change: at least 95% of contact information must be caught, and at most 5% of ordinary messages may be wrongly held. Current measured performance is **100% caught and 0% wrongly held**. Every evasion attempt found in real testing is added permanently, so the same technique cannot succeed twice.

12. Deployment and operations

Where it runs

Component	Hosting
Web application	Vercel, Tokyo region, deployed automatically on every change
Database, sign-in, files	Supabase, managed PostgreSQL
Desktop installers	Published as releases on the project repository
Android app file	The same way, once the first release is tagged

How a change reaches people

- 1 A change is committed. Automated checks run: code quality, type checking, unit tests, database rule tests, and the full journey test against a real database.
- 2 If everything passes, the web application deploys automatically.
- 3 Open browsers, desktop apps and mobile apps notice the new version within about five minutes and refresh themselves at a moment the user is not mid-sentence.

Backups and recovery

- Nightly database export covering structure and data.
- Restore procedure documented and rehearsed rather than assumed.
- Health checks and a written incident runbook.
- Log discipline verified: a staged incident must be diagnosable from logs alone, with no message content present in them.

Database changes

Every structural change is a numbered, sequential file, never edited once merged. The entire database can be rebuilt from these files alone, which is what makes the restore rehearsal meaningful. Thirty such files exist today across fifteen tables.

13. Known limits and what comes next

Stated deliberately. Every one of these is a considered decision rather than an oversight, and it is better for a client to hear them here than to discover one mid-pilot.

Current limits

Limit	Impact and plan
Images are not read	A phone number written inside a screenshot or photograph passes through untouched. This is realistically the most likely way to bypass the system. Solving it requires text recognition on images; not yet scheduled.
File contents are not read	A number inside a document passes through. Filenames are screened. Planned for the next phase.
Notifications on the Android app while closed	Requires an integration with Google's messaging service — free, roughly two to three days of work. The browser-installed version already handles this correctly.
Conversation outside the platform	If two people are already speaking on a phone call, nothing here can prevent an exchange. This is accepted by design: the product makes bypassing high-friction and evidenced, not impossible.
Voice and video calls	Not included in this phase, so no exposure through them yet. A later phase adds calls together with transcript screening.

Planned next

- **An intelligent screening pass** — a language model reviewing what the current rules cannot judge: intent, unusual phrasing, and disguises no pattern anticipates. The existing rules stay as the fast path and the fallback.
- **File and image content screening**, closing the largest current gap.
- **Notifications for the Android app while closed.**
- **Project milestones and payments**, keeping the commercial relationship on the platform as well as the conversation.
- **Voice and video calls** with transcript screening.

THE HONEST SUMMARY FOR A CLIENT

Confide makes sharing contact details inside the platform difficult, visible and permanently recorded. It does not make private contact between two determined people impossible — nothing can. What it does is remove the casual route, catch the deliberate one, and leave evidence in both cases. That is what makes a non-circumvention agreement something you can act on rather than something you merely signed.

Confide — full project documentation · 28 August 2026 · Phase 1 complete, 68 of 70 planned tasks delivered. Figures in this document are measured from the current codebase, not estimated.